

Como tudo funciona junto — o fluxo completo do SAP-1

Este é o documento que **junta as peças**: o barramento, a história de uma instrução do início ao fim, como o controlador decide, e um exercício pra você montar e traçar sozinho. Se você entender este documento, entendeu o SAP-1.

Pré-requisito: dá uma olhada em `componentes.md` (o que é cada bloco) e `palavra_controle.md` (os 12 sinais). Aqui a gente **conecta tudo**.

Parte 1 — O barramento e os "enables" (a base de tudo)

Um fio só para todo mundo

Todos os blocos estão ligados no **mesmo fio de 8 bits**: o **barramento W**. É como uma sala onde todos podem falar e ouvir — mas com uma regra:

Só UM bloco pode FALAR (colocar valor) no barramento por vez. Um ou mais podem OUVIR (copiar) ao mesmo tempo.

Se dois falassem juntos, os valores colidiriam (curto). Por isso o controlador tem o cuidado de ligar **exatamente um "falante"** em cada estado.

Quem pode falar × quem pode ouvir

Sinal	Tipo	Bloco	Ação
Ep	fala	PC	joga o endereço no barramento
~CE	fala	RAM	joga o dado lido no barramento
~Ei	fala	IR	joga o operando (endereço) no barramento
Ea	fala	Acumulador	joga A no barramento
Eu	fala	ALU	joga o resultado no barramento
~Lm	ouve	MAR	copia o barramento
~Li	ouve	IR	copia o barramento
~La	ouve	Acumulador	copia o barramento
~Lb	ouve	Registrador B	copia o barramento
~Lo	ouve	Saída	copia o barramento

(`Cp` = incrementa o PC e `Su` = modo da ALU não usam o barramento.)

Como é implementado de verdade (o mux)

No `sap1_top.v`, o barramento é um **multiplexador com prioridade** — não um tri-state. Só o "falante" habilitado passa:

```
w_bus = ep      ? {4'b0, pc_out}      : // PC falando
      (~ce)     ? ram_data            : // RAM falando
      (~ei)     ? {4'b0, ir_operand}  : // IR falando
      ea        ? acc_out             : // A falando
      eu        ? alu_out             : // ALU falando
      8'b0;                                     // ninguém -> 0
```

Agora a palavra de controle faz sentido: cada estado escolhe **um bit de "falar"** e **um ou mais de "ouvir"**. É só isso que uma instrução faz — mover valores pelo barramento, um passo por vez.

Parte 2 — A história completa de uma instrução

Vamos seguir o **ADD 11** do programa 3×4, do primeiro ao último estado.

Situação inicial: o `LDA 11` anterior já rodou, então **A = 3**. O PC aponta para o `ADD 11` (endereço 1). E `mem[11] = 3`.

Em cada estado: quem **fala**, quem **ouve**, e o **valor no barramento**.

Estado	Fala	Ouve	Barramento	Efeito
T1	PC (<code>Ep</code>)	MAR (<code>~Lm</code>)	<code>01</code>	$MAR \leftarrow 1$
T2	—	— (PC conta)	<code>00</code>	$PC \leftarrow 2$
T3	RAM (<code>~CE</code>)	IR (<code>~Li</code>)	<code>1B</code>	$IR \leftarrow \text{"ADD 11"}$
T4	IR operando (<code>~Ei</code>)	MAR (<code>~Lm</code>)	<code>0B</code>	$MAR \leftarrow 11$
T5	RAM (<code>~CE</code>)	B (<code>~Lb</code>)	<code>03</code>	$B \leftarrow 3$
T6	ALU (<code>Eu</code>)	A (<code>~La</code>)	<code>06</code>	$A \leftarrow 3+3 = 6$

Lendo em português:

1. **T1 — "onde está a instrução?"** O PC vale 1. `Ep` liga o PC no barramento (aparece `01`); `~Lm` manda o MAR copiar. Agora **MAR = 1**.
2. **T2 — "adianta o PC."** `Cp` incrementa: **PC = 2** (já aponta pro próximo). O barramento não é usado.

3. **T3 — "lê a instrução."** O MAR (=1) faz a RAM entregar `mem[1] = ADD 11 ( 0x1B )`. `~CE` põe isso no barramento; `~Li` manda o IR copiar. Agora o **IR** tem a instrução, e o controlador enxerga `opcode = ADD`.
4. **T4 — "onde está o dado?"** O IR tem o operando 11. `~Ei` joga o 11 no barramento (0B); `~Lm` faz o MAR copiar. Agora **MAR = 11**.
5. **T5 — "pega o dado."** O MAR (=11) faz a RAM entregar `mem[11] = 3`. `~CE` põe no barramento (03); `~Lb` manda o **B** copiar. Agora **B = 3**.
6. **T6 — "soma!"** A ALU está sempre calculando `A + B = 3 + 3 = 6`. `Eu` põe esse 6 no barramento; `~La` manda o **A** copiar. Agora **A = 6**.

Fim: **A foi de 3 para 6**, gastando exatamente 6 estados. O PC já aponta pro próximo `ADD`, e o ciclo recomeça.

Note como **T1-T3 são sempre iguais** (buscar a instrução) e **T4-T6 mudam** conforme o opcode. É essa a divisão busca/execução.

Parte 3 — Como o controlador decide tudo isso

Quem ligou os enables certos em cada estado? O **controlador/sequenciador**. Ele tem duas partes:

(a) O contador de anel — "onde estamos"

Um registrador que percorre os estados em ordem, avançando na **borda de descida** do clock:

```
IDLE → T1 → T2 → T3 → T4 → T5 → T6 → T1 → ...
```

É só isso: um contador que dá voltas. Não depende de dado nenhum.

(b) A lógica combinacional — "o que fazer aqui"

Uma tabela (`case`) que, dado o **estado** e o **opcode**, produz a palavra de controle:

```
case (estado)
  T1: con = ...Ep, ~Lm...           // igual pra toda instrução
  T3: con = ...~CE, ~Li...
  T6: case (opcode)
    ADD: con = ...~La, Eu...        // A ← A+B
    SUB: con = ...~La, Eu, Su...    // A ← A-B
  endcase
endcase
```

A ideia-chave: a palavra de controle depende **só** do estado e do opcode — **nunca** do dado. Por isso o processador é **determinístico**: mesmo estado + mesmo opcode = mesmos sinais, sempre. É isso que faz ele "funcionar sozinho", sem ninguém mandar.

(Detalhes da FSM, HLT e reset em `FSM_explicacao.md`.)

Parte 4 — Exercício: monte e trace A = 5 + 2

Agora **você**. Vamos escrever um programa que calcula $5 + 2$ e mostra 7.

Passo 1 — planejar

Precisamos: carregar 5, somar 2, mostrar, parar. E guardar os dados (5 e 2) em algum endereço.

```
LDA 14 ; A ← 5    (o 5 está no endereço 14)
ADD 15 ; A ← 5+2  (o 2 está no endereço 15)
OUT    ; mostra 7
HLT    ; para
```

Passo 2 — montar (traduzir pra binário)

Lembre: cada palavra é `[opcode(4) | operando(4)]`. Opcodes: `LDA=0000` `ADD=0001` `OUT=1110` `HLT=1111`.

end	instrução	opcode	operando	
0	LDA 14	0000	1110	0000_1110
1	ADD 15	0001	1111	0001_1111
2	OUT	1110	0000	1110_0000
3	HLT	1111	0000	1111_0000
14	dado = 5	—	—	0000_0101
15	dado = 2	—	—	0000_0010

Passo 3 — traçar na mão (prever o resultado)

Siga só o acumulador A:

Instrução	A antes	A depois
LDA 14	0	5
ADD 15	5	7
OUT	7	7 (mostra 7)
HLT	7	7 (para)

Resultado esperado: **saída = 7** (0x07), travado em T4 com HLT.

Passo 4 — conferir na simulação

Cole no bloco de programa do `ram_16x8.v`:

```
mem[0] = 8'b0000_1110; // LDA 14
mem[1] = 8'b0001_1111; // ADD 15
mem[2] = 8'b1110_0000; // OUT
mem[3] = 8'b1111_0000; // HLT
mem[14] = 8'd5;
mem[15] = 8'd2;
```

Ajuste `RESULTADO_ESPERADO = 8'd7` em `tb_model/tb_sap1.v`, rode `do wave_sap1.do` e veja o `acc_out` fazer `0 → 5 → 7`. Se bateu com o seu traço à mão, **você entendeu de verdade**.

Desafio extra: mude `mem[15]` para 3 e preveja o resultado **antes** de simular.
(Resposta: 8.)

O modelo mental (resumo de tudo)

O SAP-1 é uma **máquina de estados** que, a cada passo (T1–T6), gera uma **palavra de controle** que escolhe **um bloco pra falar** e **um pra ouvir** no **barramento**. Uma instrução é só uma sequência fixa de 6 desses passos, movendo valores de um registrador para outro. O programa na RAM decide *quais* instruções, mas *como* cada uma funciona é sempre a mesma receita — ditada pelo estado e pelo opcode.

Se essa frase fizer sentido pra você, fechou.